

data_ta

Contains two sets of data: a profiling/training set, and a set for the attack itself.

1. The profiling_keys.uint8.bin file contains 6,000 encryption keys (6,000 x 16 bytes)
2. The profiling_plaintext.uint8.bin file contains 6,000 plaintexts (6,000 x 16 bytes)
3. The profiling_traces.double.bin file contains 6,000 consumption traces, each consisting of 5,000 samples in double format, i.e. floating point with double precision (6,000 x 5,000 x 8 bytes)
4. The file attack_plaintext.uint8.bin contains 20 plaintexts (20 x 16 bytes)
5. The file attack_traces.double.bin contains 20 consumption traces, each consisting of 5.000 samples in double format, i.e. floating point with double precision (20 x 5.000 x 8 bytes)
6. The data are stored in "binary format", little-endian. This is the format you expect if you are working on a classic PC (Intel, AMD).

```
In [ ]: from pathlib import Path
import numpy as np

# Profiling:
# 5000 samples [float64]
# 6000 traces

data_dir: Path = Path.cwd() / 'data_ta'

ptrace_length = 5000 # samples
pr_ptraces = np.fromfile(data_dir / 'profiling_traces.double.bin', dtype=np.float64).reshape(-1, ptrace_length)
print(f'Profiling power traces shape: {pr_ptraces.shape} [{pr_ptraces.dtype}]')

pt_length = 16 # Bytes
pr_plaintexts = np.fromfile(data_dir / 'profiling_plaintext.uint8.bin', dtype=np.uint8).reshape(-1, pt_length)
print(f'Profiling plaintexts shape: {pr_plaintexts.shape} [{pr_plaintexts.dtype}]')

key_length = 16 # Bytes
pr_keys = np.fromfile(data_dir / 'profiling_keys.uint8.bin', dtype=np.uint8).reshape(-1, key_length)
print(f'Profiling keys shape: {pr_keys.shape} [{pr_keys.dtype}]')

assert pr_ptraces.shape[0] == pr_plaintexts.shape[0] == pr_keys.shape[0]

# trace length same as profiled traces
at_ptraces = np.fromfile(data_dir / 'attack_traces.double.bin', dtype=np.float64).reshape(-1, ptrace_length)
print(f'Attack power traces shape: {at_ptraces.shape} [{at_ptraces.dtype}]')

at_plaintexts = np.fromfile(data_dir / 'attack_plaintext.uint8.bin', dtype=np.uint8).reshape(-1, pt_length)
print(f'Attack plaintexts traces shape: {at_plaintexts.shape} [{at_plaintexts.dtype}]')
```

Profiling power traces shape: (6000, 5000) [float64]
Profiling plaintexts shape: (6000, 16) [uint8]
Profiling keys shape: (6000, 16) [uint8]
Attack power traces shape: (20, 5000) [float64]
Attack plaintexts traces shape: (20, 16) [uint8]

```
In [3]: SBox = np.array([
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16,
], dtype=np.uint8)
```

```
In [ ]: def init_list_of_lists(n_lists):
    return [ [] for _ in range(n_lists) ]

def calc_intermediate_value(val, key_guess):
    return SBox[val ^ key_guess]

def calc_leakage_function(val):
    return bin(val).count('1') # Hamming Weight

def partition_traces(pr_ptraces, pr_plaintexts, pr_keys, key_byte_idx):
    n_partitions = 9
    partitions = init_list_of_lists(n_partitions)

    for pr_trace, pr_pt, pr_key in zip(pr_ptraces, pr_plaintexts, pr_keys):
```

```

        v = calc_intermediate_value(pr_pt[key_byte_idx], pr_key[key_byte_idx])
        L = calc_leakage_function(v) # L (leakage value) is an integer between [0, 8]
        assert L < n_partitions
        partitions[L].append(pr_trace)

    return partitions

def calc_mean_traces(partitioning):
    mean_traces = [] # n_partitions-count (9) mean traces
    for i, partition in enumerate(partitioning):
        mean_traces.append(
            np.mean(np.stack(partition), axis=0) # the average trace (columns squished together)
        )
    return mean_traces

def select_poi(mean_traces, n_pois: int):
    # calculate sum of differences for the partitioned traces
    sod = np.zeros(ptrace_length) # sum of differences trace (ptrace_length long)
    for i, m_trace_i in enumerate(mean_traces):
        for j, m_trace_j in enumerate(mean_traces):

            if i != j:
                sod = sod + np.abs(m_trace_i - m_trace_j)
    # select n points of interest with largest sum of differences
    return np.flip(np.argsort(sod))[0:n_pois]

def comp_gauss_params(partitioning, mean_traces, pois):
    # select points of interest only
    restricted_mean_traces = [ m_trace[pois] for m_trace in mean_traces ]

    # noises = [
    #     [noise trace of each trace in partition_0], # each len(noises[i][j]) == len(pois)
    #     ...
    #     [noise trace of each trace in partition_N]
    # ]
    noises = init_list_of_lists(len(partitioning))
    assert len(partitioning) == len(restricted_mean_traces)

    for partition, m_trace, part_noise in zip(partitioning, restricted_mean_traces, noises):
        for ptrace in partition:
            part_noise.append(ptrace[pois] - m_trace)

    gauss_params = []
    # compute noise covariances between all points of interest
    for noise_partition in noises:
        gauss_params.append(
            np.cov(
                np.stack(noise_partition).T
            )
        )

    # each gaussian parameters is a symmetric noise covariance matrix
    return gauss_params

```

Profiling phase

```

In [5]: partitioning = partition_traces(pr_ptraces, pr_plaintexts, pr_keys, key_byte_idx=0)

mean_traces = calc_mean_traces(partitioning)
pois = select_poi(mean_traces, n_pois=5)
gauss_params = comp_gauss_params(partitioning, mean_traces, pois)

templates = dict()
for i in range(9):
    templates[i] = (mean_traces[i][pois], gauss_params[i])

```

Attack phase

```

In [6]: from scipy.stats import multivariate_normal

def calc_matching_p_scipy(trace, template):
    """Alternative: Calculate the log-likelihood using scipy's built-in multivariate_normal."""
    mean_vec = template[0]
    cov_mtx = template[1]

    mvn = multivariate_normal(mean=mean_vec, cov=cov_mtx)

    return mvn.logpdf(trace)

def calc_matching_p(trace, template):
    """Calculate probability the trace matches the template."""
    n_pois = len(trace)

    mean_vec = template[0]
    cov_mtx = template[1]

```

```

inv_cov_mtx = np.linalg.inv(cov_mtx)
det_cov_mtx = np.linalg.det(cov_mtx)

nom = np.exp( - 1/2 * (trace - mean_vec) @ inv_cov_mtx @ (trace - mean_vec).T )
denom = np.sqrt( ( 2 * np.pi) ** n_pois) * det_cov_mtx )
return np.log( nom / denom )

```

```

In [7]: def attack(matching_f, n_traces: int):
        P_sums = np.zeros(256)
        at_pts = at_plaintexts[0:n_traces]
        at_ptrcs = at_ptraces[0:n_traces]

        for key_guess in range(256):
            for at_pt, at_trace in zip(at_pts, at_ptrcs):
                L = calc_leakage_function(calc_intermediate_value(at_pt[0], key_guess))
                template = templates[L]
                P = matching_f(at_trace[pois], template)
                P_sums[key_guess] += P

        return P_sums, np.argmax(P_sums)

```

```

In [11]: real_key = 0x2b

for n_traces in range(1, at_plaintexts.size):
    P_sums_scipy, key_scipy = attack(calc_matching_p_scipy, n_traces)
    P_sums, key = attack(calc_matching_p, n_traces)

    if key == real_key:
        print(f'Found key: {hex(key)} using {n_traces} attack traces [my matching func]')

    if key_scipy == real_key:
        print(f'Found key: {hex(key_scipy)} using {n_traces} attack traces [scipy matching func]')

    if key_scipy == key == real_key:
        break

```

Found key: 0x2b using 4 attack traces [my matching func]
Found key: 0x2b using 4 attack traces [scipy matching func]

```

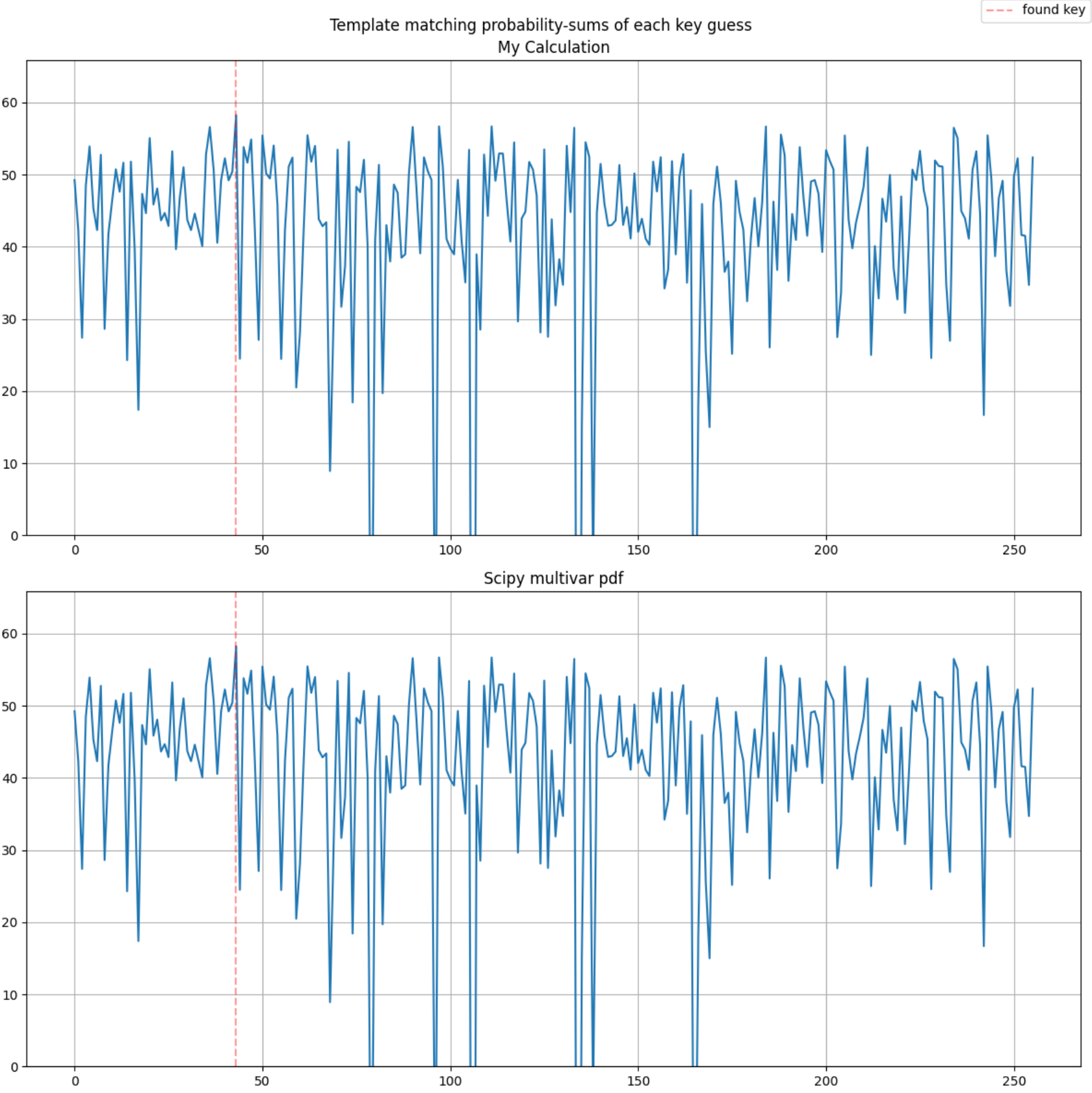
In [10]: import matplotlib.pyplot as plt

fig, axs = plt.subplots(2, figsize=[12,12])
fig.suptitle('Template matching probability-sums of each key guess')
axs[1].plot(P_sums_scipy)
axs[1].axvline(key_scipy, color='r', alpha=0.4, linestyle='--', label='found key')
axs[1].set_ylim(0)
axs[1].set_title('Scipy multivar pdf')
axs[1].grid(True)

axs[0].plot(P_sums)
axs[0].axvline(key, color='r', alpha=0.4, linestyle='--')
axs[0].set_ylim(0)
axs[0].set_title('My Calculation')
axs[0].grid(True)

fig.legend()
fig.tight_layout()

```



Note:

Though it doesn't cause any issues here, my calculation could be improved by staying in log space and utilizing numpy better to avoid numerical instabilities.